

Git Troubleshooting Reference

Understanding Staging vs. Working Directory Deletions

Scenario / Terminal Output

```
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   dishapatani1
        new file:   dishapatani2
        new file:   dishapatani3

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    dishapatani1
        deleted:    dishapatani2
        deleted:    dishapatani3
        deleted:    file1
        deleted:    file2
```

Root Cause Analysis

1 Files Were Staged (Added) First

You ran `git add dishapatani1 dishapatani2 dishapatani3`. Git added these files to the staging area (index), which is why they show up in **green** under "Changes to be committed".

2 Files Were Physically Deleted via Linux Command

After staging, you deleted the files physically using the Linux command `rm` (or deleted `file1` and `file2`) before running `git commit`.

3 Why Git Shows Both States

Git remembers your initial intent to commit the newly staged files, but when scanning your disk, it noticed the physical files are missing. Thus, it tracks both the staged addition (**green**) and the un-staged physical deletion (**red**).

Solutions

Option A: Unstage Everything & Start Clean

If you want to cancel adding these files completely and remove them from Git's staging area:

```
git restore --staged .
```

Option B: Restore the Deleted Files

If you deleted the files by mistake and want Git to bring them back into your working directory:

```
git restore .
```